

AI Virtual Try-On Web Application

Complete Technical Roadmap: React + Node.js/Express + MongoDB + FastAPI + Flow-Style-VTON

1. Executive Summary & Production Stack

This document defines the strict, milestone-driven development roadmap for building an AI-powered Virtual Try-On (VTON) web platform. The system allows users to upload a portrait photo and a garment image (or select from a clothing catalog), performs automated background isolation and warping, and synthesizes a photorealistic virtual try-on image using the peer-reviewed **Flow-Style-VTON (CVPR 2022)** model. All catalog items, user session metadata, and try-on history are managed persistently via **MongoDB**.

Component	Technology	Architectural Role & Scope
Frontend	React (Vite) + Tailwind CSS	Responsive client UI, drag-and-drop uploads, catalog browser, live try-on preview, before/after slider.
Backend Gateway	Node.js + Express.js	API Gateway, request validation, Multer file handling, forwarding to AI service, error guardrails.
Database	MongoDB + Mongoose	Persistent storage for catalog garments, category indices, user sessions, and try-on history logs.
AI Microservice	FastAPI + Uvicorn (Python)	Asynchronous REST service running inside Google Colab, executing tensor preprocessing and inference.
AI Core Model	Flow-Style-VTON (PFAFN)	Parser-Free Appearance Flow Warping Module (AFWM) + ResUNet Generator using pretrained weights.
Preprocessing	rembg (U^2-Net) + OpenCV	Automated garment background removal, binary silhouette mask generation, 256x192 bicubic resizing.
Compute / Tunnel	Google Colab (T4 GPU) + ngrok	Free cloud GPU inference accessible by the local backend via secure reverse HTTPS tunnel.

2. System Architecture & MongoDB Data Models

The architecture is organized into a clean 3-tier decoupling: the **Client Layer** interacts with the **Application Gateway (Node.js/Express)**, which communicates with **MongoDB** for metadata and proxies computationally heavy image payloads across a secure **ngrok tunnel** to the **FastAPI GPU Worker**.

MongoDB Collection	Field Definitions	Purpose / Use Case
garments	_id : ObjectId name : String (required) category : String (Shirts/Tees/Dresses) imageUrl : String (path/URL) maskUrl : String (pre-extracted mask) tags : [String] inStock : Boolean createdAt : Date	Stores curated apparel catalog items so users can select garments without uploading. Pre-computed masks eliminate preprocessing latency.

tryons	_id: ObjectId sessionId: String (UUID/IP) personImageUrl: String garmentId: ObjectId (ref: Garment, optional) customGarmentUrl: String (optional) resultImageUrl: String latencyMs: Number status: Enum ['success', 'failed'] createdAt: Date	Tracks try-on execution history, execution times, success rates, and allows users to revisit recent try-on results.
--------	---	---

3. Detailed Phase-by-Phase Development Roadmap

Milestone 0: Workspace Architecture & Guardrails [Difficulty: Low]

Objective: Establish the isolated multi-folder architecture surrounding existing repos without touching stock model code.

Exact Tasks: Audit local Node.js and Python environments; define .gitignore for checkpoints and temporary files; create clean subdirectories: /ai_service, /backend, /frontend, /docs.

Success Criteria: Directories initialized; stock repositories remain untouched.

Milestone 1: Google Colab T4 GPU Environment Setup [Difficulty: Low]

Objective: Spin up a verified cloud GPU environment capable of running PyTorch CUDA operations.

Exact Tasks: Create Colab notebook; select T4 GPU runtime; verify torch.cuda.is_available() is True; clone Flow-Style-VTON repo; install gdown, opencv, pillow, rembg.

Success Criteria: Colab environment running with 15GB Tesla T4 GPU verified.

Milestone 2: Pretrained Checkpoints Download & Benchmark Inference [Difficulty: Medium]

Objective: Acquire official weights and verify baseline model execution before writing custom code.

Exact Tasks: Download PFAFN_warp_epoch_101.pth and PFAFN_gen_epoch_101.pth via gdown; download sample VITON dataset; patch base_dataset.py (transforms.Scale to transforms.Resize); run test.py.

Success Criteria: Official test.py completes with 0 errors and generates try-on images in our_t_results/.

Milestone 3: Decoupled Single-Pair Inference Engine [Difficulty: Medium]

Objective: Create a standalone Python function that accepts raw images directly without dataset folder structures.

Exact Tasks: Write engine.py; load AFWM warp network and ResUnetGenerator into GPU VRAM once on startup; implement tryon_single_pair(person_pil, cloth_pil, mask_pil) returning a PIL Image.

Success Criteria: Function executes single try-on pair in <150ms on GPU.

Milestone 4: Automated Preprocessing & Garment Masking [Difficulty: Medium]

Objective: Enable arbitrary user photos to be used by automating background removal and aspect ratio normalization.

Exact Tasks: Build preprocessing.py; person pipeline: EXIF orientation correction, 3:4 aspect ratio crop, 256x192 bicubic resize, [-1, 1] normalization; garment pipeline: rembg U^2-Net background removal, alpha-to-mask conversion, cloth x mask zeroing.

Success Criteria: Arbitrary smartphone photos of people and clothes successfully converted to required tensors.

Milestone 5: FastAPI AI Microservice & Colab ngrok Tunnel [Difficulty: Medium]

Objective: Expose the inference engine over an HTTPS REST API for external web application consumption.

Exact Tasks: Build server.py using FastAPI and Uvicorn; endpoints: GET /health and POST /api/v1/tryon; integrate pyngrok to expose port 8000 via a secure public URL; add warm-up inference.

Success Criteria: Public ngrok HTTPS URL accepts curl POST with images and returns PNG try-on result in <1.5s.

Milestone 6: MongoDB Connection & Mongoose Data Models [Difficulty: Low]

Objective: Establish persistent data storage for apparel catalog and try-on execution history.

Exact Tasks: Install mongoose and dotenv in backend; configure MongoDB connection string (local MongoDB or free MongoDB Atlas); create Garment and TryOnHistory schemas; write seed script to populate sample clothing.

Success Criteria: MongoDB connects cleanly; garments collection seeded with 8+ sample garments.

Milestone 7: Node.js Express Gateway & API Proxy [Difficulty: Medium]

Objective: Build the application gateway to manage catalog APIs, validate uploads, and proxy inference to Colab.

Exact Tasks: Initialize Express app with CORS and Multer; create routes: GET /api/health, GET /api/garments (reads MongoDB), GET /api/history, POST /api/tryon (saves to MongoDB and proxies to Colab); handle timeouts.

Success Criteria: Postman request to http://localhost:5000/api/tryon returns generated try-on and logs record in MongoDB.

Milestone 8: React (Vite) Web Application UI [Difficulty: Medium]

Objective: Develop the modern, responsive web user interface.

Exact Tasks: Scaffold React app with Vite and Tailwind CSS; create Navbar, PersonUpload (drag-and-drop + camera), GarmentSelector (MongoDB catalog grid + custom upload tab), ActionButton, LoadingOverlay, ResultViewer (with Before/After slider).

Success Criteria: React UI runs on port 5173 with smooth previews and responsive layout.

Milestone 9: Full End-to-End System Integration [Difficulty: Medium]

Objective: Wire all layers together into a unified, seamless user experience.

Exact Tasks: Create api.js in React to call Node.js backend; bind catalog selection to try-on requests; display live Before/After comparison; save completed try-ons to MongoDB and display in recent history drawer.

Success Criteria: User uploads photo in React -> Node -> Colab GPU -> Result displayed on screen in <2 seconds.

Milestone 10: System Hardening, Error Handling & Guardrails [Difficulty: Low]

Objective: Ensure rock-solid reliability against bad inputs, oversized files, and network drops.

Exact Tasks: Add file type (JPEG/PNG) and size (<10MB) guards; add Colab disconnect detection with user-friendly alert banners; implement automatic temp file cleanup in Node.

Success Criteria: System handles invalid uploads and network disconnects gracefully without crashing.

Milestone 11: Benchmark Testing & Performance Validation [Difficulty: Low]

Objective: Empirically test latency, fidelity, and reliability across diverse test cases.

Exact Tasks: Run 20 diverse test pairs (different poses, lighting, garment types); record mask extraction time, warping time, generator time, and total roundtrip latency; document metrics.

Success Criteria: Benchmark report with latency averages and visual output quality logged.

Milestone 12: Deployment & Final Verification [Difficulty: Low]

Objective: Prepare system for final college demonstration and production readiness.

Exact Tasks: Package backend and frontend startup scripts (npm run dev); verify MongoDB Atlas cloud persistence; verify backup demo video recording in case of campus Wi-Fi failure.

Success Criteria: One-command startup; verified live demonstration ready for project evaluation.

4. Step-by-Step Antigravity Implementation Prompts

Copy and paste these exact prompts into Antigravity/Gemini sequentially. Each prompt enforces rigorous guardrails: inspect before changing code, do not break existing files, and stop on error.

PROMPT M1: Google Colab GPU Environment

```
Act as the lead AI engineer for my Virtual Try-On college project.
OBJECTIVE: Setup a clean Google Colab T4 GPU environment for Flow-Style-VTON.
INSTRUCTIONS:
1. Write Python cells for Google Colab verifying PyTorch CUDA availability and printing GPU name/VRAM.
2. Clone https://github.com/SenHe/Flow-Style-VTON.git into /content/Flow-Style-VTON.
3. Install dependencies: gdown, opencv-python-headless, pillow, matplotlib, rembg.
4. DO NOT download weights or write web services yet.
SUCCESS: Colab verifies Tesla T4 GPU active and repository cloned.
```

PROMPT M2: Download Weights & Benchmark Test

```
Act as the lead AI engineer for my Virtual Try-On college project.
OBJECTIVE: Download official Flow-Style-VTON pretrained checkpoints and run benchmark test.
INSTRUCTIONS:
1. Download PFAFN_warp_epoch_101.pth and PFAFN_gen_epoch_101.pth from Google Drive (Folder ID:
1hunG-84GOSq-qviJRvkXeSMFgnIt0TTU) to /content/checkpoints/.
2. Verify both checkpoint files exist and exceed 100MB in size.
3. Download sample test dataset (ID: 1Y7uV0gomwWyxCvvH8TibY7D9cTAUy6om) and extract it.
4. Patch base_dataset.py replacing deprecated torchvision transforms.Scale with transforms.Resize.
5. Run test.py with downloaded checkpoints and verify try-on images are saved in our_t_results/.
SUCCESS: Official test script completes with 0 errors and generates visual try-on results.
```

PROMPT M3: Decoupled Single-Pair Inference Engine

```
Act as the lead AI engineer for my Virtual Try-On college project.
OBJECTIVE: Create a standalone Python inference module engine.py that accepts single person + garment pairs.
INSTRUCTIONS:
1. Inspect Flow-Style-VTON-main/test/test.py, models/afwm.py, and models/networks.py.
2. Load AFWM and ResUnetGenerator into GPU VRAM once on module import.
3. Implement tryon_single_pair(person_pil, cloth_pil, mask_pil) -> PIL.Image.
4. Preprocess inputs to [1, 3, 256, 192] and [1, 1, 256, 192] tensors normalized to [-1.0, 1.0].
5. Execute warp and generator forward passes with torch.no_grad().
6. Return composite try-on image as RGB PIL Image. DO NOT touch original repo files.
SUCCESS: Single-pair execution returns a try-on image in <200ms on GPU.
```

PROMPT M4: Automated Preprocessing & Masking Pipeline

Act as the lead AI engineer for my Virtual Try-On college project.

OBJECTIVE: Build preprocessing.py to automate garment background removal and portrait resizing.

INSTRUCTIONS:

1. Person Pipeline: Correct EXIF orientation, crop/pad to 3:4 aspect ratio, bicubic resize to 256x192, normalize to [-1, 1].
2. Garment Pipeline: Remove background via rembg (U²-Net), convert alpha to binary silhouette mask, zero non-cloth pixels, resize to 256x192.
3. Provide fast OpenCV thresholding fallback for clean white-background studio garments.
4. Integrate preprocessing.py with engine.py.

SUCCESS: Passing unconstrained smartphone photos produces required tensors and a valid try-on image.

PROMPT M5: FastAPI Service & ngrok Colab Tunnel

Act as the lead software architect for my Virtual Try-On college project.

OBJECTIVE: Wrap engine and preprocessing into a FastAPI microservice in Colab exposed via ngrok.

INSTRUCTIONS:

1. Create server.py using FastAPI and Uvicorn.
2. Endpoints: GET /health (GPU status, VRAM) and POST /api/v1/tryon (multipart form with person and garment files, returns PNG bytes).
3. Integrate pyngrok to establish a secure public HTTPS tunnel to port 8000.
4. Implement warm-up inference on startup to eliminate cold-start latency.

SUCCESS: Public ngrok URL returns generated try-on image in response to curl POST within 1.5 seconds.

PROMPT M6: MongoDB Connection & Mongoose Data Models

Act as the lead backend engineer for my Virtual Try-On college project.

OBJECTIVE: Setup MongoDB connection and Mongoose data models in /backend.

INSTRUCTIONS:

1. Install mongoose and dotenv in /backend.
 2. Create db.js to handle connection to MongoDB (using process.env.MONGODB_URI).
 3. Create models/Garment.js: name, category, imageUrl, maskUrl, tags, inStock, createdAt.
 4. Create models/TryOnHistory.js: sessionId, personImageUrl, garmentId, resultImageUrl, latencyMs, status, createdAt.
 5. Create scripts/seedGarments.js to populate database with 8 sample garments across categories.
- SUCCESS: MongoDB connects cleanly and seed script populates sample garments successfully.

PROMPT M7: Node.js / Express API Gateway

Act as the lead backend engineer for my Virtual Try-On college project.

OBJECTIVE: Build the Express API Gateway in /backend.

INSTRUCTIONS:

1. Initialize Express app with CORS, dotenv, and Multer for multipart form uploads.
2. Configure AI_SERVICE_URL in .env to point to the Colab ngrok tunnel.
3. Routes: GET /api/health (checks gateway + Colab status), GET /api/garments (fetches from MongoDB), POST /api/tryon (receives person upload + garment, proxies to Colab, logs result in MongoDB).
4. Ensure temporary upload files are cleaned up after inference.

SUCCESS: Postman POST to http://localhost:5000/api/tryon returns generated image and logs entry in MongoDB.

PROMPT M8: React (Vite) Web Application

Act as the lead frontend engineer for my Virtual Try-On college project.

OBJECTIVE: Build the React web frontend in /frontend using Vite and Tailwind CSS.

INSTRUCTIONS:

1. Scaffold Vite + React application with Tailwind CSS and Lucide React.
 2. Components: Navbar (title, status badge), PersonUpload (drag-drop, preview), GarmentSelector (MongoDB catalog grid + custom upload tab), ActionButton (Try On), LoadingOverlay (progress spinner), ResultViewer (Before/After comparison slider, download button).
 3. Connect API calls to Node.js backend (http://localhost:5000/api).
- SUCCESS: Web app runs on port 5173 with complete interactive state management and visual previews.

PROMPT M9: End-to-End System Integration & History

Act as the lead full-stack engineer for my Virtual Try-On college project.
OBJECTIVE: Integrate React frontend, Node backend, MongoDB, and Colab AI service.
INSTRUCTIONS:
1. Wire Try-On button to send multipart POST request through Node to Colab.
2. Display synthesized result image on the Before/After comparison canvas.
3. Add a Recent Try-On History drawer in the UI that fetches records from GET /api/history.
4. Display toast notifications for success and clear error messages on failure.
SUCCESS: Complete roundtrip: upload in browser -> GPU inference -> MongoDB log -> screen render in <2s.

PROMPT M10: System Hardening & Validation Guardrails

Act as the lead QA engineer for my Virtual Try-On college project.
OBJECTIVE: Implement error guardrails across frontend and backend.
INSTRUCTIONS:
1. Client-side guards: validate file format (JPEG/PNG only) and size cap (<10MB).
2. Backend guards: Multer error handling, 30-second timeout threshold for Colab requests.
3. Disconnect handling: if Colab tunnel is unreachable, show clear banner instructing user to start Colab GPU.
SUCCESS: App handles oversized files, wrong formats, and network disconnections gracefully without crashing.

PROMPT M11: Performance Benchmarking

Act as the lead AI researcher for my Virtual Try-On college project.
OBJECTIVE: Benchmark the complete virtual try-on system across 20 test pairs.
INSTRUCTIONS:
1. Write benchmark script testing 20 diverse person + garment combinations.
2. Record: rembg preprocessing latency (ms), warping latency (ms), generator latency (ms), total roundtrip time (ms).
3. Generate summary markdown table in docs/benchmarks.md with averages and percentiles.
SUCCESS: Comprehensive benchmark report with empirical latency and success rates generated.

PROMPT M12: Deployment & Final Verification

Act as the lead release engineer for my Virtual Try-On college project.
OBJECTIVE: Finalize project package and verify demonstration readiness.
INSTRUCTIONS:
1. Verify npm run dev starts both backend and frontend cleanly.
2. Ensure MongoDB Atlas connection string is documented in backend/.env.example.
3. Document 3-step startup instructions in root README.md.
SUCCESS: Flawless end-to-end execution ready for final project demonstration.

5. Milestone Execution Checklist

Status	Milestone	Deliverable / Goal
☐ M0	Workspace Architecture	Clean folder layout (/ai_service, /backend, /frontend, /docs).
☐ M1	Colab GPU Environment	T4 GPU verified, PyTorch/CUDA active, repo cloned.
☐ M2	Checkpoints & Benchmark	PFAFN weights downloaded; official test.py runs with 0 errors.
☐ M3	Single-Pair Engine	engine.py executes single pair in <150ms on GPU.
☐ M4	Automated Preprocessing	rembg background removal and 256x192 resizing pipeline.
☐ M5	FastAPI & ngrok Tunnel	Public HTTPS API running in Colab returning PNG try-on stream.
☐ M6	MongoDB Setup & Schemas	Mongoose Garment and TryOnHistory schemas; seed data loaded.
☐ M7	Node.js Express Gateway	Express API gateway proxying try-on requests to Colab.
☐ M8	React (Vite) Frontend	UI with upload zone, catalog grid, Before/After slider.
☐ M9	End-to-End Integration	Full roundtrip: Upload -> Node -> MongoDB -> GPU -> UI display.
☐ M10	Hardening & Error Guards	File size guards, timeout handling, Colab disconnect alerts.
☐ M11	Performance Benchmarking	Latency metrics recorded across 20 test pairs in docs/.
☐ M12	Final Verification	One-command system startup; live demonstration verified.